

AP Calculus & Precalculus Adaptive Learning Platform

Complete Technical & Product Overview

Platform Documentation

May 2026

Contents

1	Executive Summary	5
2	Platform Architecture	6
2.1	Monorepo Structure	6
2.2	Technology Stack	6
2.3	Environments and Deployment	7
3	The Admin Application	8
3.1	Problem Generation and Authoring	8
3.1.1	The Generate Page (<code>/generate</code>)	8
3.1.2	Problem Data Model	9
3.2	RAG Examples Pool	10
3.2.1	What RAG Examples Are	10
3.2.2	The RAG Examples Page (<code>/rag-examples</code>)	10
3.2.3	The RAG Agent Page (<code>/rag-agent</code>)	10
3.3	The Keyword Engine	11
3.3.1	What Keywords Are	11
3.3.2	The Keywords Page (<code>/keywords</code>)	12
3.3.3	The Keyword Test Page (<code>/keyword-test</code>)	12
3.3.4	The Keyword Dedup Page (<code>/keyword-dedup</code>)	12
3.3.5	The Compare Page (<code>/compare</code>)	12
3.4	The Supervised Tagging Pipeline (<code>/tagging</code>)	12

4	The Student Application	14
4.1	Authentication and Session Management	14
4.1.1	Custom Username/Password Auth	14
4.1.2	The Student Session Record	14
4.2	The Free Practice Portal (Home Page)	15
4.2.1	Layout and UX	15
4.2.2	Problem Selection Algorithm	15
4.2.3	Attempt Recording and Strength Updates	16
4.3	The Adaptive Learn Flow	17
4.3.1	Phase State Machine	17
4.3.2	Phase 1: Diagnostic	17
4.3.3	Phase 2: Diagnostic Results and Routing	17
4.3.4	Phase 3a: Full Lesson	18
4.3.5	Phase 3b: Refresher	19
4.3.6	Phase 3c: Targeted Practice	19
4.3.7	Phase 4: Mastery Quiz	19
4.3.8	Phase 5: Adaptive Practice (/learn/practice)	19
4.3.9	On-Demand Content Generation	20
5	AI Pipeline	21
5.1	Problem Generation Models	21
5.2	Prompt Engineering Architecture	21
5.2.1	The Format Contract	21
5.2.2	RAG-Augmented Generation	21
5.2.3	Assessment System Prompt	22
5.2.4	Keyword Tagger Prompt	22
5.3	Embedding-Based Semantic Search	22
6	Database Design	24
6.1	Overview	24
6.2	Core Tables	24
6.2.1	problems	24
6.2.2	rag_examples	25
6.2.3	topic_metadata	25

6.3	Student Tables	25
6.3.1	student_accounts	25
6.3.2	student_sessions	26
6.3.3	student_problem_attempts	26
6.4	Keyword Taxonomy Tables	26
6.4.1	learn_categories	26
6.4.2	learn_keywords	26
6.5	Learn System Tables	27
6.5.1	learn_lessons	27
6.5.2	learn_refreshers	27
6.5.3	learn_tips	27
6.5.4	learn_diagnostic_problems	28
6.5.5	learn_student_keyword_states	28
7	Adaptive Learning Algorithms	30
7.1	The Strength Model	30
7.1.1	Practice Strength Updates	30
7.1.2	Diagnostic Strength Updates	30
7.2	Problem Selection and Scoring	30
7.2.1	Keyword Weakness Score	31
7.2.2	Difficulty Match Score	31
7.2.3	Rating Score	31
7.2.4	Combined Score and Selection	31
7.3	Dynamic Difficulty Calibration	31
8	Content Rendering	33
8.1	KaTeX Rendering Pipeline	33
8.2	Interactive Visualization Tags	33
8.2.1	FunctionGraph	33
8.2.2	SlopeField	33
8.2.3	Parsing and Rendering	34
8.3	LaTeX Format Conventions	34

9	End-to-End Data Flows	35
9.1	A Student’s First Practice Session	35
9.2	A Student Taking the Learn Flow	35
9.3	An Admin Creating a Batch of Precalculus Problems	35
10	What the Platform Produces for Students	37
10.1	A Personalized Skill Map	37
10.2	Adaptive Problem Difficulty	37
10.3	Structured Instruction When Needed	37
10.4	Multiple Layers of Feedback	37
10.5	Mastery Verification	37
10.6	Problem Quality at Scale	38
11	Summary	39

1 Executive Summary

This document provides a thorough technical and product description of an adaptive mathematics learning platform designed for AP Calculus AB and precalculus students. The platform is composed of two web applications — an admin authoring tool and a student-facing practice portal — backed by a shared PostgreSQL database, an AI-powered content generation pipeline, and a keyword-based adaptive learning engine.

The platform’s goal is to give every student a personalized, self-paced path from initial diagnosis of their skill level through targeted instruction and on to exam-ready mastery. It does this by combining:

- A curated problem bank generated and quality-checked by AI
- A fine-grained keyword taxonomy that tags every problem with the specific skills it tests
- A real-time strength model that tracks each student’s competency at the keyword level
- An adaptive delivery system that chooses problems at the right difficulty for each student at each moment
- A multi-phase learn flow (diagnostic → lesson/refresher → mastery quiz → adaptive practice) that routes students to exactly the instruction they need

The system is fully functional as a web application and requires no installation by students. It runs in a browser, persists state across sessions, and serves as a complete self-study companion for high school and college-level mathematics.

2 Platform Architecture

2.1 Monorepo Structure

The codebase is organized as a Turborepo monorepo with the following top-level layout:

```
ap-calc-platform/  
  apps/  
    admin/          (port 3001 -- teacher/admin authoring tool)  
    student/        (port 3002 -- student practice portal)  
  packages/  
    types/          (shared TypeScript types)  
    supabase/        (singleton Supabase client)  
    constants/       (AP Calc topic catalog, keyword taxonomy)  
  supabase/  
    migrations/      (all database migrations, versioned SQL)  
  scripts/           (seed scripts for topics, keywords, content)
```

Both apps are built with **Next.js 15** (React 19, App Router) and use TypeScript throughout. Turborepo manages parallel builds, dev server startup, and lint across all packages. The shared `packages/` layer ensures that topic definitions, keyword taxonomy, and core types are never duplicated between the two apps.

2.2 Technology Stack

Layer	Technology
Frontend	Next.js 15, React 19, Tailwind CSS, KaTeX
Backend	Next.js API routes (Node.js runtime)
Database	Supabase (PostgreSQL) with Row Level Security
AI / LLM	OpenAI <code>gpt-5.4-mini</code> (generation, assessment, tagging)
AI / Embeddings	OpenAI <code>text-embedding-3-small</code> (1536-dim vectors)
Problem rendering	KaTeX (math), custom FunctionGraph + SlopeField components
Expression eval	<code>expr-eval</code> (safe math expression parsing for graphs)
Batch generation	OpenAI Batch API (JSONL upload → async job → result download)
Auth (student)	Custom username/bcrypt (no Supabase Auth)
Monorepo tooling	Turborepo, npm workspaces

2.3 Environments and Deployment

Both apps share a single `.env.local` file at the repository root that supplies:

```
NEXT_PUBLIC_SUPABASE_URL      # Supabase project URL
NEXT_PUBLIC_SUPABASE_ANON_KEY # Public anon key (read-only content access)
SUPABASE_SERVICE_ROLE_KEY     # Full DB access for API routes and seed scripts
OPENAI_API_KEY                # Required for all AI features
```

The student app's API routes also need `OPENAI_API_KEY` directly, so `apps/student/.env.local` must duplicate that variable. Both apps can run simultaneously in development: `npm run dev` starts both via Turborepo's parallel task runner.

3 The Admin Application

The admin app (running on port 3001) is the authoring and management tool used by teachers or curriculum designers. It handles all problem creation, quality review, keyword tagging, and content seeding. Students never interact with it directly.

3.1 Problem Generation and Authoring

3.1.1 The Generate Page (/generate)

The primary authoring interface for creating new AP Calculus AB problems. It supports both **multiple-choice questions (MCQ)** and **free-response questions (FRQ)**.

Workflow from the admin’s perspective:

1. Select one or more AP Calc AB topics from a searchable dropdown (populated from the 60+ canonical topics in `packages/constants/topics.json`)
2. Set difficulty (1–5 slider) and question type (MCQ or FRQ)
3. Click “Generate” — the UI calls `POST /api/generate-problem`
4. The server returns a rendered problem with LaTeX math, answer choices (MCQ) or rubric (FRQ), and a difficulty badge
5. The admin can **Refine** with free-text feedback, **Regenerate** from scratch, or **Save** to the database

What happens server-side during generation:

The generation pipeline in `apps/admin/app/api/generate-problem/route.ts` is a multi-step process:

Step 1 — RAG Retrieval (create mode only). The server samples 6 keywords from the unit-specific keyword taxonomy for the selected topics. It then queries both the `problems` table and the `rag_examples` table looking for approved problems whose `keyword_weights` overlap with those sampled keywords. Up to 6 matching problems are selected and injected into the prompt as gold-standard reference examples — this technique (Retrieval-Augmented Generation) steers the model toward the stylistic and structural conventions already established in the problem bank.

Step 2 — Prompt Construction. The system builds format-specific prompts from `apps/admin/lib/ai/prompts.ts`:

- For MCQ: a random “emphasis topic” is selected from the topic pool, and a difficulty narrative (“straightforward procedural application,” “requires multi-step reasoning,” etc.) is injected
- For FRQ: a random archetype (TYPE A through TYPE G) is selected — these map to different FRQ styles (area/volume, rates, accumulation, graph analysis, etc.)
- All prompts enforce LaTeX conventions: prose inside `\text{}`, math expressions outside, `\\` for line breaks in aligned environments, no implicit multiplication in expression evaluators

Step 3 — LLM Call. The server calls `gpt-5.4-mini` with `response_format: { type: "json_object" }` to ensure structured output. The model returns a JSON object with `latex_content`, `solution_latex`, `choices`, `correct_index`, and `difficulty`.

Step 4 — Post-Assessment. After generation, the server calls a second LLM pass via `POST /api/assess-problem`. The assessor model:

- Checks for KaTeX rendering issues (invalid syntax, broken commands)
- Evaluates content quality (is the problem well-formed? Is the difficulty appropriate? Are the distractors meaningful?)
- Returns `rendering_issues` and `content_issues` strings
- Returns a `difficulty` integer that may differ from the model’s self-reported difficulty

If the assessor finds rendering or content issues, the server automatically fires a **refinement pass** — it sends the problem and the assessor’s feedback back to the generator with a refine-mode prompt, gets a revised problem, and returns that to the admin. This auto-refinement happens transparently with no admin interaction required.

Step 5 — Save. When the admin clicks Save, the problem is inserted into the `problems` table with `status = 'approved'` and `estimated_difficulty` seeded from the assessed difficulty. A fire-and-forget async call is made to `autoTagKeywords()` which sends the problem content to `gpt-5.4-mini` for keyword tagging (see Section 5).

3.1.2 Problem Data Model

Every problem in the `problems` table has the following core fields:

Field	Type	Purpose
<code>id</code>	UUID	Primary key
<code>latex_content</code>	TEXT	The problem stem, fully LaTeX-formatted
<code>solution_latex</code>	TEXT	Complete worked solution in LaTeX
<code>choices</code>	JSONB	Array of 4 choice strings (MCQ only)
<code>correct_index</code>	INTEGER	Index (0–3) of the correct choice
<code>difficulty</code>	INTEGER 1–5	Human/model-assigned static difficulty
<code>estimated_difficulty</code>	NUMERIC	EMA-calibrated from student attempts
<code>keyword_weights</code>	JSONB	Map of keyword → weight (GIN indexed)
<code>type</code>	TEXT	'multiple_choice' or 'free_response'
<code>status</code>	TEXT	'draft' or 'approved'
<code>avg_rating</code>	NUMERIC	Running average of 1–5 student ratings
<code>rating_count</code>	INTEGER	Number of ratings received
<code>attempt_count</code>	INTEGER	Total student attempts

Field	Type	Purpose
<code>success_count</code>	INTEGER	Correct attempts (for success rate)
<code>flag_count</code>	INTEGER	Times students have reported the problem

The `keyword_weights` column has a GIN index which enables efficient `?` (key overlap) queries — this is what makes it fast to filter thousands of problems down to only those relevant to a student’s selected topics.

3.2 RAG Examples Pool

3.2.1 What RAG Examples Are

The `rag_examples` table stores a curated set of “gold-standard” problems used as generation seeds. They are not the main problem bank — they exist purely to give the LLM high-quality structural and stylistic examples to imitate. When the generator retrieves 6 examples for its prompt, it draws from both `problems` and `rag_examples`.

3.2.2 The RAG Examples Page (`/rag-examples`)

This page lets admins promote approved problems from the main bank into the RAG seed pool, or directly add examples. Each RAG example can carry:

- `keyword_weights` — used for keyword-overlap retrieval
- `difficulty` — difficulty label
- `notes` — a human-readable note about what makes this a good example
- `course` — `'ap_calc'` or `'precalc'` (examples are course-specific)
- `promoted_problem_id` — if this example was promoted from a `problems` row, this FK links back to it

3.2.3 The RAG Agent Page (`/rag-agent`)

This is the bulk precalculus problem generation tool. The workflow is:

1. **Upload a PDF** — the admin uploads a curriculum PDF (e.g., a precalculus practice blueprint). The server extracts text using `pdf-parse`, then sends it to `gpt-5.4` to extract a structured list of { `name`, `description` } problem types. Hierarchical documents (sections with sub-items) are flattened so each individual skill becomes its own problem type entry.
2. **Review problem types** — the extracted list is displayed as a checklist. The admin can select/deselect types and set a per-type count (number of problems to generate at varying difficulties).

3. **Try a few interactively** — before committing to batch generation, the admin can review problems one at a time. Each problem goes through the full generate → assess → auto-refine pipeline via `POST /api/rag-agent/generate-one`. The admin sees the full rendered preview, including the problem stem, solution, choices, and a descriptions card showing `problem_description` (what skill the problem tests) and `wrong_answer_descriptions` (what mistake leads to each incorrect answer).
4. **Approve or skip** — approved problems are saved to `rag_examples` via `POST /api/rag-examples`. The system auto-tags keywords on save and displays them as colored badges on already-approved problems.
5. **Send remaining to batch** — once the admin is satisfied with quality, clicking “Batch remaining N” submits all unreviewed tasks to the **OpenAI Batch API**:
 - The server builds a JSONL file with one chat completion request per task
 - The JSONL file is uploaded to OpenAI Files
 - A batch job is created with `completion_window: "24h"`
 - The server returns a `batchId`
 - The UI enters a polling phase, checking `GET /api/rag-agent/batch-status?batchId=...` every 30 seconds
 - When the batch completes, the server downloads the output file, parses each JSONL result line, applies LaTeX sanitization, and returns all generated problems for review
6. **Batch review** — each batch result is displayed one at a time for approve/skip review, identical to the interactive flow.

Why batch? The Batch API charges 50% less than real-time calls and is appropriate for large-volume, non-time-critical generation. For 1,000 problems at `gpt-5.4-mini` pricing, batch generation is dramatically cheaper than interactive mode.

3.3 The Keyword Engine

3.3.1 What Keywords Are

The keyword taxonomy is a three-tier classification system designed to describe exactly what skills and contexts a math problem involves:

Tier 1: Content Keywords (207 total) — Specific mathematical skills. Examples: `product_rule_exponents`, `quotient_rule_exponents`, `zero_exponent`, `evaluating_piecewise`, `domain_of_rational_function`. These are organized into 9 content categories (`exponents_and_radicals`, `functions`, `function_transformations`, `inverse_functions`, `piecewise_functions`, `polynomials`, `rational_functions`, `exponential_and_logarithmic_functions`, `trigonometry`).

Tier 2: Action Tags (14 total) — What the student does: `solve`, `simplify`, `evaluate`, `graph_or_sketch`, `factor`, `identify`, `interpret`, `write_equation`, etc.

Tier 3: Representation/Style Tags (21 total) — How the problem is presented or what pedagogy it represents: `graph_given`, `table`, `symbolic_expression`, `multiple_choice`, `computational`, `conceptual`, `real_world_context`, `multi_step`, etc.

All 242 keywords + 9 categories are stored in the `learn_keywords` and `learn_categories` tables with descriptions and 1536-dimensional embedding vectors generated by `text-embedding-3-small`.

3.3.2 The Keywords Page (/keywords)

Displays all 9 content categories with approved keyword counts. For each category, admins can:

- View all approved keywords with their descriptions
- Generate new candidate keywords via AI (sends the category description to `gpt-5.4-mini`, returns a list for review without saving)
- Approve/reject individual keywords from the candidate list
- Delete existing keywords
- Trigger embedding generation for any keywords missing vectors

3.3.3 The Keyword Test Page (/keyword-test)

An interactive testing tool. Paste any problem text and click “Analyze” — the server:

1. Embeds the problem text using `text-embedding-3-small`
2. Fetches all approved keywords and categories from the database
3. Computes cosine similarity between the problem embedding and every keyword embedding
4. Returns: top 15 content keywords (5 anchored from the best-matching category + 10 global), top 10 action/representation tags, and top 15 categories

This lets admins verify that the keyword system is classifying problems correctly before it’s used in automated tagging.

3.3.4 The Keyword Dedup Page (/keyword-dedup)

Finds near-duplicate keywords using pairwise cosine similarity. The admin sets a threshold (e.g., 0.92) and the server computes similarity for all keyword pairs within a category. Pairs above the threshold are displayed with their scores so admins can decide which to keep and which to delete.

3.3.5 The Compare Page (/compare)

A simple two-input tool: paste two text strings (could be two keywords, two problems, or any text), and the server embeds both and returns their cosine similarity score. Useful for manually checking whether two descriptions are semantically close enough to be considered duplicates.

3.4 The Supervised Tagging Pipeline (/tagging)

This page exposes a full retrieve-then-rerank tagging pipeline for any problem. It is a local admin tool for understanding and auditing how the automated keyword tagging works.

Input: A problem with its question, solution, description, and per-choice metadata (text, whether correct, wrong-answer description).

Pipeline execution (all parallelized where possible):

1. **Batch embed** — A single `text-embedding-3-small` API call embeds:
 - The concatenated `question + solution + description` string (problem-level embedding)
 - Each `wrong_answer_description` separately (one embedding per wrong answer)
2. **Fetch taxonomy** — All 207 content keywords, 35 tags, and 9 categories are fetched from the database in parallel. Each row includes its stored embedding vector.
3. **Cosine similarity in JavaScript** — Every taxonomy row is scored against the problem embedding and each wrong-answer embedding. No external vector database is needed — all computation happens in-process in a single JS loop.
4. **Candidate slates** — The top-N highest-similarity rows are collected:
 - Problem level: top 10 categories, top 25 content keywords, top 12 tags
 - Each wrong answer: top 20 content keywords, top 10 tags (scored against the wrong-answer embedding, not the overall problem embedding)
5. **LLM reranking** — Parallel `gpt-5.4-mini` calls, one for the problem and one per wrong answer. Each call receives only the retrieved candidates (never the full taxonomy), asks the model to select the most applicable labels and assign weights between 0 and 1.
6. **Enforcement** — Any keyword ID the LLM returns that was not in the candidate slate is silently filtered out. This prevents hallucinated keyword names.

Output: The UI displays all intermediate results — retrieval texts, cosine similarity scores for every candidate, raw LLM JSON, and final reranked labels with weights. This transparency lets admins understand exactly why a problem was tagged the way it was.

Design rationale: The two-stage retrieve-then-rerank architecture exists because the full taxonomy (242 keywords + 9 categories) is too large to fit reliably in a single LLM prompt. Cosine similarity pre-filters to relevant candidates, and the LLM reranks and weights them. Critically, wrong answers are retrieved and tagged independently using their own descriptions — this captures the specific misconception each distractor targets, which is valuable for adaptive practice (the student’s wrong choice tells you which skill gap to address).

4 The Student Application

The student app (port 3002) is the primary learning interface. It has two main areas: a **free-practice portal** (the home page) and a **learn flow** (/learn and /learn/practice).

4.1 Authentication and Session Management

4.1.1 Custom Username/Password Auth

The student app uses a custom authentication system rather than Supabase's built-in auth. This was a deliberate choice to keep the student experience simple (no email confirmation, no OAuth, just a username and password).

Registration (POST /api/auth/register): Creates a new `student_sessions` row (the student's state container) and a new `student_accounts` row linking the session UUID to the username and bcrypt password hash.

Login (POST /api/auth/login): Verifies the bcrypt hash. On success, returns the `sessionId` from the linked `student_sessions` row.

Session storage: After login, three values are written to `localStorage`: - `ap_calc_student_session_id` — the UUID that identifies this student's session in all API calls - `ap_calc_account_id` — the account UUID (used as an auth guard on page load) - `ap_calc_username` — displayed in the UI header

API authentication pattern: Every API call from the student app includes `sessionId` in the POST body. There are no cookies, no Authorization headers. The server looks up the `student_sessions` row by that UUID to read or update the student's state.

4.1.2 The Student Session Record

The `student_sessions` table is the central state container for each student:

Field	Content
<code>id</code>	The UUID stored in <code>localStorage</code>
<code>strengths</code>	Topic-level strength map (legacy, still maintained)
<code>keyword_strengths</code>	Keyword-level strength map — the primary strength model
<code>created_at</code> / <code>updated_at</code>	Timestamps

`keyword_strengths` is a JSONB map like { "product_rule_exponents": 0.73, "zero_exponent": 0.41, ... }. Every answer a student submits updates this map using the bounded EMA algorithm described in Section 7.

4.2 The Free Practice Portal (Home Page)

4.2.1 Layout and UX

The home page is a two-panel layout:

Left sidebar: A scrollable topic selector organized by AP Calc AB units (Unit 1 through Unit 8). Each topic has: - A checkbox for selection/deselection - The topic name - A color-coded horizontal strength bar showing the student's current strength on that topic (red → orange → green as strength improves from 0 to 1)

Unit headers act as group toggles (select/deselect all topics in a unit at once). An “All / None” control handles full selection.

Below the topic list, a difficulty control lets students choose between “Recommended” (algorithmically matched to their current level) and “Custom” (a 1–5 button selector).

Main panel: The problem area. States: - Idle: welcome message, “Start Practice” button - Loading: spinner - Answering: problem stem + 4 answer choices as clickable buttons - Revealed: correct answer highlighted in green, wrong answer in red, full solution shown below

After revealing the solution, students can rate the problem (1–5 stars) and report it if something seems wrong. A “Next Problem →” button loads the next problem.

4.2.2 Problem Selection Algorithm

When the student clicks “Start Practice” or “Next Problem,” `POST /api/next-problem` runs the following decision sequence:

1. Determine target difficulty. If the student chose “Recommended,” the server computes:

```
targetDifficulty = 1 + average_keyword_strength * 3
```

where `average_keyword_strength` is the mean strength across keywords relevant to the selected topics. This maps the student's strength range [0, 1] to a difficulty range [1, 4], targeting roughly 75% success rate — challenging but achievable.

If the student chose a custom difficulty, that value overrides the computation.

2. Score all unseen candidate problems. The server fetches all approved MCQ problems from the database, filters out problems the student has already seen (by checking `student_problem_attempts`), further filters out flagged or poorly-rated problems, and filters to problems whose `keyword_weights` overlap with the selected topics' keyword set.

Each remaining problem is scored by a three-factor formula:

```
score = kwScore ** diffScore ** ratingScore
```

- **kwScore** — keyword-weakness score: how much the problem's keywords overlap with the student's weakest keywords. A problem testing skills the student is weak in gets a higher score.

- **diffScore** — Gaussian centered on `targetDifficulty` with `sigma=1`: problems near the target difficulty score highest, problems far away score near zero.
- **ratingScore** — $0.4 + 0.6 \cdot (\text{avg_rating} / 5)$: high-rated problems get a modest boost; unrated problems receive 0.8 (slightly above average, assuming quality until proven otherwise).

3. Select from top-K with weighted randomness. The top 8 scoring problems are placed in a weighted random pool. The final selection is a weighted random pick from this pool (not always the top scorer). This adds exploration, ensuring students see variety rather than the same problem repeatedly.

4. Fallback to RAG examples. If no relevant unseen problems exist in the main bank, the server checks `rag_examples` for any relevant unseen examples. If found, it automatically “promotes” the RAG example into the `problems` table as an approved row, records the link, and serves it. Subsequent requests will use the promoted row.

5. Fallback to generation. If neither the problem bank nor the RAG pool has a suitable problem, the server calls `POST /api/generate-problem` on the admin app, runs an assessment, keyword-tags the result, saves it to the database, and serves it marked “Newly Generated” in the UI.

4.2.3 Attempt Recording and Strength Updates

When a student answers a problem, `POST /api/record-attempt` handles:

1. **Upsert** the attempt into `student_problem_attempts` (unique per session+problem, so re-answering updates the record rather than duplicating it)
2. **Keyword strength update** via bounded EMA:
 - Correct: `strength[k] += 0.12 * w * (1 - strength[k])`
 - Wrong: `strength[k] -= 0.12 * w * strength[k]`
 - Only keywords that appear in the “strength-tracked” tier of the taxonomy are updated (not formatting/tag-only keywords)
3. **Dynamic difficulty calibration** via EMA:
 - Compute student’s current skill level: `skill = 1 + weighted_avg_strength * 4 → [1, 5]`
 - Set difficulty target: `correct ? skill - 0.5 : skill + 0.5`
 - Update: `estimated_difficulty = clamp(old + 0.15 * (target - old), 1, 5)`

This means every approved problem’s `estimated_difficulty` is a living estimate that converges toward its true difficulty as more students attempt it. New problems start with their static difficulty as the seed, and over time the estimated value reflects actual student performance data.

4.3 The Adaptive Learn Flow

The `/learn` route implements the platform’s most sophisticated feature: a full adaptive instruction sequence that takes a student from zero knowledge to confirmed mastery of a specific skill.

The current implementation is built around the **exponent rules** topic, but the data model and state machine are fully generalized to any keyword in the taxonomy.

4.3.1 Phase State Machine

`loading` → `diagnostic` → `results` → [`lesson` | `refresher` | `mastery_quiz`] → `practice` (redirect)

The learn page maintains a **phase** state variable that determines which UI component is rendered. Transitions between phases are driven by diagnostic results and user interactions.

4.3.2 Phase 1: Diagnostic

The student sees 5 multiple-choice questions fetched from `learn_diagnostic_problems` for the topic. These are not ordinary problems — they are specifically curated for diagnostic purposes and carry two keyword weight maps:

- `umbrella_keywords` — weights for the broad category keyword (e.g., `exponent_rules` with weight 1.0)
- `in_depth_keywords` — weights for specific sub-skills (e.g., `product_rule_exponents: 0.8`, `zero_exponent: 0.6`)

For each question, the student does not just choose A/B/C/D — they have two additional buttons:

- **“I’ve learned this but don’t remember it”** — marks the question as “forgotten”
- **“I’ve never seen this before”** — marks the question as “never seen” and excludes it from scoring

These self-report buttons are critical because: - “Never seen” correctly prevents the EMA algorithm from penalizing the student for not knowing material they were never taught - “Forgotten” triggers a stronger decay signal (same as a wrong answer) and contributes to routing toward a refresher

No solutions are shown during the diagnostic — it’s purely a gauge of current knowledge.

4.3.3 Phase 2: Diagnostic Results and Routing

After all 5 questions are answered, the client runs the EMA algorithm locally (in `diagnosticScoring.ts`) to produce a `DiagnosticResult`:

EMA update rules:

Correct answer: `score[k] += 0.25 ** weight ** (1 - score[k])`
 Wrong answer: `score[k] -= 0.20 ** weight ** score[k]`
 Forgotten: same decay as wrong
 Never seen: excluded from EMA entirely

Both `umbrellaScores` (maps `exponent_rules` $\rightarrow$ score) and `inDepthScores` (maps each sub-skill $\rightarrow$ score) are updated independently.

Routing logic (in order of priority):

Condition	Route	Meaning
<code>neverSeenCount</code> $\geq$ 3 OR <code>umbrellaScore</code> $<$ 0.35	<code>full_lesson</code>	Genuinely new material
<code>forgottenCount</code> $\geq$ 2 OR <code>umbrellaScore</code> $<$ 0.5	<code>refresher</code>	Learned before, needs reminder
<code>umbrellaScore</code> $<$ 0.75 OR any in-depth skill $<$ 0.45	<code>targeted</code>	Mostly knows it, specific gaps
Otherwise	<code>skip</code>	Already proficient

The result is posted to `POST /api/learn/classify`, which persists keyword states to `learn_student_keyword_states` and upserts the student's scores.

The results screen shows: - A verdict card (“You’ve mastered this!” / “You need more practice” / “You need a refresher” / “You need to learn this from scratch”) - The specific weakest sub-skills (keyword names) - A progress bar for the umbrella score - A “Continue” button that transitions to the appropriate next phase

4.3.4 Phase 3a: Full Lesson

For students routed to `full_lesson`, the lesson is fetched from `GET /api/learn/lesson/[keyword]`. If no lesson exists in the database yet, it is **generated on demand** by `learnGenerator.ts`:

The generator calls `gpt-5.4-mini` with a detailed system prompt that produces a `micro_steps` array — each step contains:

- `explanation_latex` — a 1–3 sentence explanation of one sub-concept, fully in LaTeX with `\text{}` for prose
- `example_latex` — a worked example using `\begin{aligned}...\end{aligned}` with `&=` alignment and `\\` line breaks
- `check_question` — an MCQ question testing the sub-concept, with 4 choices and a solution
- `hint_latex` — a one-line tip targeting the most common mistake

The lesson renders as a **2-step micro-lesson**:

For each step: 1. Student reads the explanation and example (rendered with KaTeX) 2. Student answers the check question - **Correct**: proceeds to next step (or mastery quiz if last step) - **Wrong**: a hint appears; the student can retry

A feedback panel at the bottom lets students rate the lesson as helpful or not (stored in `learn_feedback`). After all steps are completed, lesson progress is marked complete in `learn_student_lesson_progress`.

4.3.5 Phase 3b: Refresher

For students routed to `refresher`, the refresher is fetched from `GET /api/learn/refresher/[keyword]`. On first access, it is generated on demand by `learnGenerator.ts`:

A refresher consists of: - `rule_latex` — a 1–2 sentence statement of the rule/property - `example_latex` — one concise worked example - `check_question` — a single MCQ to confirm the student can apply the refresher

The refresher UI displays: 1. Rule card 2. Example (with “Show example” toggle) 3. Check question

After the check question is answered, the student proceeds to the mastery quiz.

4.3.6 Phase 3c: Targeted Practice

For students routed to `targeted`, the system identifies the weakest in-depth keywords and routes directly to the mastery quiz targeting those specific skills. No instruction is given; it’s pure practice with harder questions focused on the identified gaps.

4.3.7 Phase 4: Mastery Quiz

After any instruction phase (lesson, refresher, or targeted), the student takes a **4-question mastery quiz**. Questions are fetched from `learn_mastery_quiz_problems` for the keyword — each at difficulty 3 or 4.

The quiz is **one question at a time** with no back-tracking: - Each question shows the problem, choices, and (after answering) the full solution - Score is tracked silently

After all 4 questions: - **Pass ($\geq 80\%$, i.e., 3+ correct)**: keyword state is updated to `mastered` in `learn_student_keyword_states`, `spaced_review_due_at` is set for future spaced repetition review, and the student is redirected to `/learn/practice` - **Fail ($< 80\%$)**: the student is also redirected to `/learn/practice`, but the keyword remains in a non-mastered state

Quiz results are persisted to `learn_mastery_quiz_results`.

4.3.8 Phase 5: Adaptive Practice (`/learn/practice`)

The practice page at `/learn/practice?keyword=&topic=` is a continuous MCQ loop:

Problem selection uses `POST /api/learn/practice/next`: 1. Fetch the student’s `learn_student_keyword_states` for this keyword 2. Compute target difficulty: $1 + \text{strength} \cdot 3 \rightarrow [1, 4]$ 3. Fetch `learn_practice_problems` for the keyword 4. Score and select using the same weighted-random mechanism as the main practice portal (but scoped to just the keyword’s problem pool)

Tip popup appears after 2 consecutive wrong answers. The tip is fetched from `GET /api/learn/tip/[keyword]`, generated on demand if absent. Tips are short (`\text{Remember:`

} or `\text{Watch out: }` + one math insight), and are rated helpful/not-helpful with aggregate counts stored on the `learn_tips` row.

Mastery offer appears after a consecutive correct streak: “You’re on a streak! Want to take the mastery quiz?” This allows strong students to skip ahead to mastery verification.

Attempt recording via `POST /api/learn/practice/attempt` updates `learn_student_keyword_states`:

- `consecutive_correct` increments on success, resets to 0 on failure
- `total_attempts` and `correct_attempts` are maintained
- `last_practiced_at` is updated

4.3.9 On-Demand Content Generation

All learn content (lessons, refreshers, tips, practice problems, mastery quiz problems) is generated once and stored permanently. The API routes check the database first; if the content is missing, they generate it, store it, and return it. Deleting a row from the database forces regeneration with a fresh prompt.

The content generation system uses carefully engineered prompts that enforce:

- Multi-step solutions use `\begin{aligned}...\end{aligned}` with `&=` alignment
- Never single-line equality chains like $x^a = x^b = x^c$
- All prose (English text) lives inside `\text{...}` — nothing is rendered as plain text in math mode
- Choices are wrapped in `$. . . $` for inline math rendering

5 AI Pipeline

5.1 Problem Generation Models

All AI calls use OpenAI’s API. The model used for generation, assessment, keyword tagging, and content creation is **gpt-5.4-mini**. Embedding calls use **text-embedding-3-small** (1536 dimensions).

gpt-5.4-mini was chosen as the default because it is substantially cheaper than **gpt-5.4** while producing format-compliant output when prompts are engineered carefully. For PDF parsing (extracting problem types from curriculum documents), **gpt-5.4** is used because document understanding benefits from the more capable model.

5.2 Prompt Engineering Architecture

5.2.1 The Format Contract

The most important prompt engineering challenge is ensuring that every generated LaTeX renders correctly in KaTeX. The prompts enforce a strict format contract:

- In a JSON string value, every LaTeX backslash must be doubled: `\text{}` in the final LaTeX becomes `\\text{}` in the JSON string
- A LaTeX line break (`\\`) becomes `\\\\` in the JSON string
- When a line break is followed by a `\text{}` command, there **MUST** be a space between them: `\\\\ \text{}` — without the space, the JSON parser interprets `\t` as a tab character (a valid JSON escape sequence), corrupting the output

This last rule is a subtle but critical JSON escape trap. The platform’s `sanitizeLatexContent()` function in `ragProblemParser.ts` includes a repair step that detects and fixes this corruption pattern even if the model violates the rule.

5.2.2 RAG-Augmented Generation

The standard AP Calc problem generator uses Retrieval-Augmented Generation:

1. Sample 6 keywords from the unit-specific keyword set for the selected topics
2. Query `problems` and `rag_examples` for approved problems whose `keyword_weights` contain any of those 6 keywords (using the GIN-indexed `?|` operator)
3. Sort results by keyword overlap score, take the top 6
4. Serialize these 6 examples into the prompt as a “format contract” block
5. Instruct the model to match these examples exactly in style, LaTeX conventions, and structural format

This grounds every new problem in the conventions already established for the problem bank, dramatically reducing format drift and LaTeX errors compared to zero-shot generation.

5.2.3 Assessment System Prompt

The assessor runs as a separate LLM call after generation. Its system prompt is `AP_CALC_ASSESS_SYSTEM` which instructs the model to check:

1. **Rendering issues:** Invalid LaTeX (commands that don't exist in KaTeX, malformed `\begin{aligned}` environments, unclosed `\text{}` blocks)
2. **Content issues:** Wrong answer key, ambiguous wording, unrealistic difficulty for the target level, distractors that are not plausibly wrong
3. **Difficulty calibration:** Return an integer 1–5 that reflects the actual cognitive demand of the problem

The assessor's output drives: - `assessedDifficulty` — stored as the problem's difficulty (overrides the generator's self-reported difficulty) - Auto-refinement trigger — if either `rendering_issues` or `content_issues` is non-null, the problem is automatically refined before being returned

5.2.4 Keyword Tagger Prompt

After a problem is saved, `autoTagKeywords()` runs as a fire-and-forget async call. The tagger is given:

- The full keyword taxonomy organized by category (from `packages/constants/keywords.json`)
- The problem's `latex_content` and `solution_latex`

It returns `{ keyword_weights: { "keyword_name": 1 } }` — a flat map of keywords $\rightarrow$ 1. The tagger is instructed to include only keywords that a student **must actively apply** to solve this problem, not background knowledge. Keywords outside the valid taxonomy set are silently rejected. Topic core keywords (from `packages/constants/unitKeywordMap.ts`) are merged in after tagging, ensuring that broad topic anchors are always present.

5.3 Embedding-Based Semantic Search

The keyword retrieval system (used in `/keyword-test`, `/tagging`, and keyword dedup) works through three steps:

1. **Embed the query** — call `text-embedding-3-small` to get a 1536-dim vector for the input text
2. **Fetch stored embeddings** — retrieve all rows with non-null `embedding` from `learn_keywords` or `learn_categories`
3. **Cosine similarity in JS** — compute dot product divided by product of norms for every pair

```
function cosine(a: number[], b: number[]): number {
  let dot = 0, normA = 0, normB = 0;
  for (let i = 0; i < a.length; i++) {
```

```

    dot += a[i] * b[i];
    normA += a[i] * a[i];
    normB += b[i] * b[i];
}
return dot / (Math.sqrt(normA) * Math.sqrt(normB));
}

```

For keyword search, the “anchored” retrieval strategy works as follows: - Find the **best-matching category** (highest cosine similarity between the problem embedding and each category embedding) - Take the top 5 keywords from that category (anchored to the most likely domain) - Take the top 10 keywords globally (regardless of category) - Deduplicate → final set of up to 15 content keywords

This anchoring prevents the global top-10 from being dominated by one category’s keywords when the best match is concentrated in a single domain.

6 Database Design

6.1 Overview

The database is a PostgreSQL instance hosted on Supabase. Row Level Security (RLS) is enabled on all tables. The policy model is:

- **service_role**: full read/write access to all tables (used by all Next.js API routes)
- **anon**: read-only access to content tables (problems, rag_examples, topic_metadata, learn_*)
- **student session tables**: anon has full access during development (intended to be tightened with proper session validation)

All migrations are versioned SQL files in `supabase/migrations/` and apply in timestamp order.

6.2 Core Tables

6.2.1 problems

The main problem bank. Stores all MCQ and FRQ problems authored through the admin generate page.

<code>id</code>	<code>UUID PRIMARY KEY</code>	
<code>latex_content</code>	<code>TEXT NOT NULL</code>	
<code>solution_latex</code>	<code>TEXT NOT NULL</code>	
<code>choices</code>	<code>JSONB</code>	<code>-- null for FRQ</code>
<code>correct_index</code>	<code>INTEGER</code>	<code>-- null for FRQ</code>
<code>rubric</code>	<code>TEXT</code>	<code>-- null for MCQ</code>
<code>difficulty</code>	<code>INTEGER 1-5</code>	
<code>estimated_difficulty</code>	<code>NUMERIC(4,2)</code>	<code>-- null until first attempt</code>
<code>keyword_weights</code>	<code>JSONB NOT NULL DEFAULT '{}'</code>	
<code>type</code>	<code>TEXT ('multiple_choice' 'free_response')</code>	
<code>status</code>	<code>TEXT ('draft' 'approved')</code>	
<code>avg_rating</code>	<code>NUMERIC</code>	
<code>rating_count</code>	<code>INTEGER DEFAULT 0</code>	
<code>attempt_count</code>	<code>INTEGER DEFAULT 0</code>	
<code>success_count</code>	<code>INTEGER DEFAULT 0</code>	
<code>flag_count</code>	<code>INTEGER DEFAULT 0</code>	
<code>topic_weights</code>	<code>JSONB</code>	<code>-- legacy, superseded by keyword_weights</code>
<code>variant_index</code>	<code>INTEGER</code>	
<code>created_at</code>	<code>TIMESTAMPTZ</code>	

The GIN index on `keyword_weights` enables ?| overlap queries that filter to relevant problems in $O(\log n)$ time rather than $O(n)$.

6.2.2 rag_examples

Gold-standard examples for RAG-augmented generation.

<code>id</code>	<code>UUID PRIMARY KEY</code>
<code>topic_id</code>	<code>TEXT</code>
<code>variant_index</code>	<code>INTEGER</code>
<code>keyword_weights</code>	<code>JSONB NOT NULL DEFAULT '{}'</code>
<code>latex_content</code>	<code>TEXT NOT NULL</code>
<code>solution_latex</code>	<code>TEXT NOT NULL</code>
<code>choices</code>	<code>JSONB</code>
<code>correct_index</code>	<code>INTEGER</code>
<code>notes</code>	<code>TEXT</code>
<code>difficulty</code>	<code>INTEGER</code>
<code>course</code>	<code>TEXT ('ap_calc' 'precalc')</code>
<code>problem_description</code>	<code>TEXT</code>
<code>wrong_answer_descriptions</code>	<code>JSONB</code>
<code>promoted_problem_id</code>	<code>UUID REFERENCES problems(id)</code>
<code>created_at</code>	<code>TIMESTAMPTZ</code>

`promoted_problem_id` is set when the student app’s problem-selection fallback promotes a RAG example into the problems table. This prevents the same content from being promoted twice and links the two rows.

6.2.3 topic_metadata

The AP Calculus AB topic catalog — 60+ topics organized by the College Board’s AP Calculus AB curriculum.

<code>id</code>	<code>TEXT PRIMARY KEY ('1_1', '1_2', ... '8_4')</code>
<code>name</code>	<code>TEXT NOT NULL</code>
<code>description</code>	<code>TEXT</code>
<code>example</code>	<code>TEXT</code>
<code>unit_name</code>	<code>TEXT</code>

Topic IDs follow the College Board’s unit/subtopic numbering scheme (e.g., `2_4` = Unit 2, Topic 4 = “Power Rule”).

6.3 Student Tables

6.3.1 student_accounts

<code>id</code>	<code>UUID PRIMARY KEY</code>
<code>username</code>	<code>TEXT UNIQUE NOT NULL</code>
<code>password_hash</code>	<code>TEXT NOT NULL (bcrypt)</code>
<code>session_id</code>	<code>UUID REFERENCES student_sessions(id)</code>
<code>created_at</code>	<code>TIMESTAMPTZ</code>

6.3.2 student_sessions

```
id                UUID PRIMARY KEY (client-generated)
strengths         JSONB DEFAULT '{}' -- legacy topic strengths
keyword_strengths JSONB DEFAULT '{}' -- primary: keyword-level strengths
created_at        TIMESTAMPTZ
updated_at        TIMESTAMPTZ
```

6.3.3 student_problem_attempts

```
id                UUID PRIMARY KEY
session_id        UUID REFERENCES student_sessions(id)
problem_id        UUID REFERENCES problems(id)
selected_index    INTEGER NOT NULL
correct           BOOLEAN NOT NULL
rating            INTEGER (1-5)
flagged           BOOLEAN DEFAULT FALSE
attempted_at      TIMESTAMPTZ
UNIQUE(session_id, problem_id)
```

The unique constraint ensures each student attempts each problem at most once in their history (re-answers update the record). This prevents duplicate data and ensures the “seen problems” filter in problem selection is accurate.

6.4 Keyword Taxonomy Tables

6.4.1 learn_categories

```
id                TEXT PRIMARY KEY -- e.g. 'exponents_and_radicals'
name              TEXT NOT NULL
description       TEXT
order_index       INTEGER
embedding         JSONB -- float[] with 1536 elements
```

6.4.2 learn_keywords

```
id                TEXT PRIMARY KEY -- snake_case e.g. 'product_rule_exponents'
name              TEXT
label            TEXT NOT NULL
description       TEXT
examples          JSONB
category_id       TEXT REFERENCES learn_categories(id)
tier              TEXT ('umbrella' | 'in_depth' | 'tag')
status            TEXT ('draft' | 'approved' | 'rejected')
embedding         JSONB -- float[] with 1536 elements
created_at        TIMESTAMPTZ
```

Tags (action + representation/style keywords) also live in this table with `tier = 'tag'` and `category_id = 'tags'`.

6.5 Learn System Tables

6.5.1 learn_lessons

```
id          UUID PRIMARY KEY
keyword_id  TEXT REFERENCES learn_keywords(id) ON DELETE CASCADE
micro_steps JSONB NOT NULL -- array of MicroStep objects
model       TEXT -- which model generated it
generated_at TIMESTAMPTZ
UNIQUE(keyword_id)
```

`micro_steps` is a JSONB array of objects:

```
[{
  "step_index": 0,
  "explanation_latex": "...",
  "example_latex": "...",
  "check_question": {
    "latex_content": "...",
    "choices": ["$...$", "$...$", "$...$", "$...$"],
    "correct_index": 1,
    "solution_latex": "..."
  },
  "hint_latex": "..."
}]
```

6.5.2 learn_refreshers

```
id          UUID PRIMARY KEY
keyword_id  TEXT REFERENCES learn_keywords(id) ON DELETE CASCADE
rule_latex  TEXT NOT NULL
example_latex TEXT NOT NULL
check_question JSONB NOT NULL -- same shape as lesson check_question
model       TEXT
generated_at TIMESTAMPTZ
UNIQUE(keyword_id)
```

6.5.3 learn_tips

```
id          UUID PRIMARY KEY
keyword_id  TEXT REFERENCES learn_keywords(id) ON DELETE CASCADE
tip_latex   TEXT NOT NULL
helpful_count INTEGER DEFAULT 0
```

```

not_helpful_count INTEGER DEFAULT 0
model            TEXT
generated_at     TIMESTAMPTZ
UNIQUE(keyword_id)

```

6.5.4 learn_diagnostic_problems

```

id                UUID PRIMARY KEY
topic_id          TEXT NOT NULL
latex_content     TEXT NOT NULL
choices           JSONB NOT NULL
correct_index     INTEGER NOT NULL
difficulty        INTEGER (1-5)
umbrella_keywords JSONB NOT NULL DEFAULT '{}'
in_depth_keywords JSONB NOT NULL DEFAULT '{}'
diagnostic_purpose  TEXT
order_index       INTEGER DEFAULT 0
created_at        TIMESTAMPTZ

```

umbrella_keywords and in_depth_keywords are the weight maps used by the diagnostic scoring EMA.

6.5.5 learn_student_keyword_states

```

id                UUID PRIMARY KEY
session_id        UUID REFERENCES student_sessions(id)
keyword_id        TEXT NOT NULL
topic_id          TEXT NOT NULL
state             TEXT DEFAULT 'unknown'
                  CHECK IN ('unknown', 'needs_lesson', 'needs_refresher',
                             'needs_practice', 'in_progress', 'mastered')
umbrella_score    NUMERIC(4,3) DEFAULT 0.5
in_depth_score    NUMERIC(4,3) DEFAULT 0.5
confidence        NUMERIC(4,3) DEFAULT 0.0
consecutive_correct INTEGER DEFAULT 0
total_attempts    INTEGER DEFAULT 0
correct_attempts  INTEGER DEFAULT 0
clicked_never_seen BOOLEAN DEFAULT FALSE
clicked_forgot    BOOLEAN DEFAULT FALSE
spaced_review_due_at TIMESTAMPTZ
last_practiced_at TIMESTAMPTZ
updated_at        TIMESTAMPTZ
UNIQUE(session_id, keyword_id)

```

This table is the core per-student knowledge state. The state enum tracks the student's progress through the learn flow for each keyword: `unknown` → `needs_lesson` → (after lesson)

→ `in_progress` → (after mastery quiz pass) → `mastered`. The `spaced_review_due_at` field supports future spaced repetition features.

7 Adaptive Learning Algorithms

7.1 The Strength Model

The platform uses an **Exponential Moving Average (EMA)** to model student competency at the keyword level. Unlike simple right/wrong tallies, the EMA gives more weight to recent performance and bounds the strength value in $[0, 1]$.

7.1.1 Practice Strength Updates

After each answered problem in the practice portal:

```
alpha = 0.12
Correct: strength[k] += alpha ** weight[k] ** (1 - strength[k])
Wrong:   strength[k] -= alpha ** weight[k] ** strength[k]
```

- `weight[k]` is the keyword’s weight in the problem’s `keyword_weights` map (typically 1.0 for primary keywords)
- The `(1 - strength)` term ensures correct answers have diminishing effect near the ceiling — impossible to reach 1.0
- The `strength` term ensures wrong answers have diminishing effect near the floor — impossible to reach 0.0

Only keywords in the “strength-tracked” tier of the taxonomy are updated. Tag-only keywords (format/style descriptors) are excluded because they don’t represent learnable skills.

7.1.2 Diagnostic Strength Updates

The diagnostic EMA uses different learning rates to reflect the different quality of diagnostic evidence:

```
Correct: score[k] += 0.25 ** weight ** (1 - score[k])    (more confident evidence)
Wrong:   score[k] -= 0.20 ** weight ** score[k]          (slightly asymmetric)
Forgotten: same as Wrong
Never Seen: excluded entirely
After lesson: score[k] += 0.15 ** weight ** (1 - score[k]) (reading != mastery)
```

The higher learning rate (0.25 vs 0.12) reflects the fact that diagnostic questions are specifically chosen to probe competency, whereas practice problems are chosen to challenge rather than diagnose.

7.2 Problem Selection and Scoring

The practice problem selection algorithm combines three independent factors:

7.2.1 Keyword Weakness Score

$$\text{kwScore} = \sum_k (\text{weight}[k] ** (1 - \text{strength}[k])) / \sum_k \text{weight}[k]$$

A problem whose keywords are all at strength 0.0 (no knowledge) gets a kwScore of 1.0. A problem whose keywords are all at strength 1.0 (fully mastered) gets a kwScore of 0.0. This naturally deprioritizes problems the student has already mastered and prioritizes problems in the student's weakest areas.

7.2.2 Difficulty Match Score

$$\text{diffScore} = \exp(-0.5 * (\text{effectiveDifficulty} - \text{targetDifficulty})^2)$$

This is a Gaussian curve centered on the target difficulty with standard deviation 1. A problem at exactly the target difficulty gets a diffScore of 1.0; a problem 2 levels away gets a diffScore of 0.135.

`effectiveDifficulty` prefers `estimated_difficulty` over the static `difficulty` when the former is available (i.e., after at least one student has attempted the problem).

7.2.3 Rating Score

$$\text{ratingScore} = 0.4 + 0.6 ** (\text{avg_rating} / 5)$$

- Unrated problems: 0.8 (neutral-positive prior)
- Perfectly rated (5.0): 1.0
- Poorly rated (1.0): 0.52

Poorly-rated problems are never fully excluded (minimum ratingScore is 0.52), but they are consistently outcompeted by similar problems with better ratings.

7.2.4 Combined Score and Selection

$$\text{score} = \text{kwScore} ** \text{diffScore} ** \text{ratingScore}$$

Problems with `score = 0` are excluded entirely. Of the remaining scored problems, the top 8 are placed in a weighted random pool. The weighted random selection uses scores as weights, so the top-scoring problem is most likely to be chosen but is not guaranteed — this ensures students see variety.

7.3 Dynamic Difficulty Calibration

Each problem has two difficulty values: - `difficulty`: the static value assigned at creation (based on the generator's self-report and the assessor's evaluation) - `estimated_difficulty`: the EMA-calibrated value updated by student performance

After each student attempt:

```
skill = 1 + weighted_avg_keyword_strength ** 4          → [1, 5]
target = skill - 0.5  (if correct)
        = skill + 0.5  (if wrong)
estimated_difficulty = clamp(old + 0.15 ** (target - old), 1, 5)
```

The intuition: if a student of skill level 3 gets a problem wrong, the problem is at least as hard as level 3.5 (they couldn't solve it). If they get it right, the problem is at most as easy as level 2.5. Over many attempts from many students, **estimated_difficulty** converges to the problem's true difficulty, independent of the model's initial estimate.

8 Content Rendering

8.1 KaTeX Rendering Pipeline

All mathematical content is rendered in the browser using **KaTeX** — a fast, purely client-side LaTeX renderer. The rendering pipeline is:

1. **Raw string from database** (e.g., `"\\text{Simplify } \\dfrac{x^5 \\cdot x^2}{x^3}"`)
2. **Preview component** receives the string as `latexContent` prop
3. **parseVizSegments()** splits the string into LaTeX segments and visualization tag segments (`FunctionGraph`, `SlopeField`)
4. **renderToString() from KaTeX** converts each LaTeX segment to HTML
5. The HTML is injected with `dangerouslySetInnerHTML`
6. A `useEffect` with `requestAnimationFrame` measures all `.katex-display` wrappers and reduces font size proportionally if they overflow the container (minimum 0.45em) — this prevents horizontal scrollbars on narrow screens

The `splitRawLatexByText()` function handles the two types of LaTeX content: - **Display math** (delimited by `$$...$$` or `\[...\]`): rendered as block-level - **Inline math** (delimited by `$...$` or `\(...\)`): rendered inline - **Plain text with `\text{}` wrappers**: rendered directly as KaTeX

8.2 Interactive Visualization Tags

Problem content strings can embed JSX-like self-closing tags that trigger interactive graph renderers:

8.2.1 FunctionGraph

```
<FunctionGraph equation="x^2 - 3" rangeX="-3,3" rangeY="-4,6" />
<FunctionGraph equation="x^2" equation2="2*x" rangeX="-3,3" rangeY="-4,6" />
<FunctionGraph pieces="x^2|-3,0; 2*x+1|0,3" holes="0,1" dots="0,0"
    rangeX="-3,3" rangeY="-1,7" />
```

Renders an interactive SVG graph with: - Single or dual function curves - Piecewise functions with configurable segment ranges - Open/closed circles at endpoints (`holes` = open, `dots` = filled) - Configurable X and Y axis ranges - Grid lines and axis labels - Built with `expr-eval` for safe expression evaluation (no `eval()`)

8.2.2 SlopeField

```
<SlopeField equation="y - x" rangeX="-3,3" rangeY="-3,3" />
```

Renders a slope field diagram where every grid point shows a short line segment with slope equal to $f(x, y)$ at that point. Used for differential equations problems.

8.2.3 Parsing and Rendering

`parseVizSegments()` uses a regex to find `<FunctionGraph ... />` and `<SlopeField ... />` tags in the content string, splitting the string into an array of alternating LaTeX and visualization segments. The `Preview` component maps over this array, rendering LaTeX segments with KaTeX and visualization segments with the appropriate React component.

Tags must appear on their own line after any `\end{aligned}` block — never mid-sentence or inside a math environment. This constraint is enforced in all generation prompts and is validated by the post-assessment KaTeX error checker.

8.3 LaTeX Format Conventions

All content (problems, solutions, lessons, refreshers) follows these conventions:

Element	Convention
English prose	Always inside <code>\text{...}</code>
Multi-step solutions	<code>\begin{aligned}...\end{aligned}</code> with <code>&=</code> alignment
Line breaks between steps	<code>\\</code> at end of each aligned row
Choice text (MCQ)	Wrapped in <code>...\$</code> for inline math
Fractions	<code>\dfrac{}{}</code> (display-style) for readability
Dots/multiplication	<code>\cdot</code>
Trailing period after math	Inside <code>\text{.}</code>

9 End-to-End Data Flows

9.1 A Student's First Practice Session

1. Student registers at `/login` — a `student_sessions` row and `student_accounts` row are created
2. Student lands on the home page — session UUID is loaded from `localStorage`, `student_sessions.keyword_strengths` is fetched (empty `{}` for new user)
3. Student selects topics, clicks “Start Practice” — `POST /api/next-problem` runs:
 - `keyword_strengths = {}` → all strengths default to 0.5
 - `targetDifficulty = 1 + 0.5 ** 3 = 2.5`
 - Fetches all approved MCQ problems, filters to those matching selected topics
 - Scores each by weakness + difficulty match + rating
 - Returns a difficulty-2 or difficulty-3 problem appropriate for a new student
4. Student answers — `POST /api/record-attempt` updates `keyword_strengths` in the session
5. Student continues — each subsequent `next-problem` call uses the updated strengths to find progressively better-matched problems

9.2 A Student Taking the Learn Flow

1. Student navigates to `/learn` — session loaded, `GET /api/learn/diagnostic?topic=exponent_rules` fetches 5 diagnostic questions
2. Student answers questions, clicking “I’ve never seen this” on 3 of them — `flaggedNeverSeen` is set for those answers
3. Local EMA runs: because 3 questions were never seen, `neverSeenCount = 3` → route = `full_lesson`
4. `POST /api/learn/classify` persists the keyword states
5. `GET /api/learn/lesson/exponent_rules` — no lesson exists yet → `learnGenerator.generateAndStoreLesson` is called
6. GPT generates 2 micro-steps and stores them in `learn_lessons`
7. Student reads step 1 explanation, views example, answers check question (gets it wrong, sees hint, retries, gets it right)
8. `POST /api/learn/lesson/progress` updates current step to 1
9. Student completes step 2 — lesson is marked complete
10. Mastery quiz: 4 questions fetched from `learn_mastery_quiz_problems`, student scores 3/4 (75%) — passes
11. `POST /api/learn/mastery-quiz/submit` sets `state = 'mastered'` in `learn_student_keyword_states`, sets `spaced_review_due_at`
12. Student is redirected to `/learn/practice?keyword=exponent_rules` for continued practice

9.3 An Admin Creating a Batch of Precalculus Problems

1. Admin navigates to `/rag-agent`, uploads `precalculus_blueprint.pdf`
2. `POST /api/rag-agent/parse` extracts ~45 problem types from the PDF using GPT
3. Admin reviews the list, selects all, sets count = 3 per type (135 total tasks)

4. Admin clicks “Generate” on the first task — `POST /api/rag-agent/generate-one` runs:
 - Builds precalc prompt with schema + gold-star format example
 - Calls `gpt-5.4-mini`, gets JSON back
 - Runs assessor (checks rendering + content)
 - Auto-refines once if issues found
 - Returns rendered problem
5. Admin reviews, approves — `POST /api/rag-examples` saves to DB, fires keyword tagging
6. After reviewing 5–10 problems and confirming quality, admin clicks “Batch remaining 125”
7. `POST /api/rag-agent/batch-submit` builds 125-line JSONL, uploads to OpenAI Files, creates batch job, returns `batchId`
8. UI enters polling mode, shows progress bar, polls `GET /api/rag-agent/batch-status?batchId=...` every 30 seconds
9. ~1 hour later, batch completes — server downloads output file, parses 125 result lines, applies sanitization, returns all problems
10. Admin reviews batch results one at a time — approves or skips each
11. Approved examples are available immediately in the student app’s RAG fallback pool

10 What the Platform Produces for Students

10.1 A Personalized Skill Map

Every student session maintains a live keyword-level strength profile. After enough practice, the strength bars in the sidebar reflect genuine ability differences: topics the student has mastered are green, topics with gaps are red or orange. This gives students immediate, actionable insight into where they should focus.

10.2 Adaptive Problem Difficulty

The platform never locks students into a fixed difficulty. A student who improves rapidly will automatically receive harder problems because `targetDifficulty` rises with their keyword strengths. A student who struggles will receive easier problems because `targetDifficulty` falls. This Goldilocks calibration targets a ~75% success rate — challenging enough to build skill, achievable enough to build confidence.

Furthermore, as more students use the platform, `estimated_difficulty` values on problems converge toward true difficulty based on actual student performance. The problem bank self-calibrates over time.

10.3 Structured Instruction When Needed

Rather than just throwing problems at students, the Learn flow first diagnoses exactly what the student knows, then routes them to the right type of instruction: - Complete beginners get a full micro-lesson with explanation, worked example, and a check question per sub-concept - Students who once knew the material but forgot get a compact refresher - Strong students who have specific gaps go straight to targeted practice - Fully proficient students skip to practice

This routing means students never sit through instruction they don't need, and never get dropped into practice on material they've never been taught.

10.4 Multiple Layers of Feedback

After every practice problem, students see: - Whether they were correct - The full worked solution with step-by-step reasoning - The ability to rate the problem (which improves future problem selection for all students) - The ability to flag problems that seem wrong or unclear

Lesson and refresher content has explicit helpful/not-helpful feedback buttons. Tips have the same. Over time, this feedback shapes which content is served to more students and which is deprioritized.

10.5 Mastery Verification

Rather than declaring mastery based on raw accuracy, the platform uses a formal mastery quiz with a 80% pass threshold. Passing the quiz triggers a state transition to `mastered` and sets a spaced repetition review date — when that date arrives, the platform can re-test the skill to ensure retention. This ensures that students achieve durable understanding, not just a temporary performance peak.

10.6 Problem Quality at Scale

The combination of the RAG pool, LLM assessment, auto-refinement, student ratings, and flag counts creates a quality management system that operates at the scale of hundreds of problems:

- Problems that rate poorly (below 2.0 stars with 5+ ratings) are excluded from delivery
- Problems with high flag rates ($\geq 10\%$ of attempts) are excluded
- Auto-refinement catches the most common rendering and content errors before problems ever reach students
- The RAG seed pool ensures new problems maintain stylistic consistency with the established bank

Over time, the platform's content quality improves automatically through the combination of AI generation quality checks and crowdsourced student feedback.

11 Summary

The platform is a full-stack adaptive learning system built on modern web technologies and a carefully engineered AI pipeline. Its key distinguishing characteristics are:

Keyword-level modeling: Rather than tracking performance at the coarse topic level, the platform models competency at the level of individual skills (207 content keywords). This enables precise routing, targeted instruction, and accurate problem selection.

Three-tier AI pipeline: Generation (create problems), assessment (check quality), and tagging (classify skills) are three separate AI calls that work together. This separation of concerns lets each model focus on what it does best and allows auto-refinement to catch and fix the generator’s mistakes.

Live difficulty calibration: Problem difficulties are not static labels — they are continuously updated by student performance data, converging toward empirically accurate estimates over time.

On-demand content generation: Lessons, refreshers, tips, and quiz problems do not need to be pre-authored — they are generated on first request, stored permanently, and served instantly thereafter. This makes it trivially easy to extend the platform to new topics.

Structured learning path: The diagnostic → instruction → mastery quiz → adaptive practice sequence is a complete pedagogical model, not just a quiz engine. It addresses the question “what should this student do right now?” at every stage of their learning journey.

The result is a platform that can serve a student who has never seen exponent rules and a student who has studied calculus for months, giving each of them exactly the problems and instruction they need to grow.